Sensitivity Label Taxonomy Design — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Sensitivity Label Taxonomy Design (Microsoft Purview) 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

What is delivered . . . 5

Business outcome . . . 5

Important boundaries . . . 6

1. Solution Overview — Sensitivity Label Taxonomy Design (Microsoft Purview) 6

What this solution is . . . 6

Why this matters . . . 6

The label object model (what you are designing) . . . 7

The naming convention (the core design deliverable) . . . 7

Sample taxonomy (a technical starting point — confirm with your governance owners) . . . 8

The protection / marking mapping (the second core deliverable) . . . 8

Artifact names used in every doc and script . . . 9

What you can control . . . 9

Limitations (be honest with buyers) . . . 10

The standard deployment model . . . 10

2. Requirements Guide — Sensitivity Label Taxonomy Design 10

2.1 Licensing for sensitivity labels, protection, and content marking . . . 11

2.2 Information Protection Admin role (least privilege) . . . 11

2.3 Security & Compliance PowerShell connectivity . . . 11

2.4 A classification scheme and in-scope workloads . . . 12

2.5 A pilot ring and a pilot-first commitment . . . 12

2.6 Rights-management (encryption) prerequisites . . . 12

2.7 A signed-off taxonomy and protection-mapping sheet . . . 13

Global Requirements — Microsoft Purview Solutions 13

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 14

3. Security & Compliance PowerShell + Graph toolchain . . . 14

4. Data sources & scope in scope . . . 14

5. Compliance, records, privacy & legal stakeholders . . . 15

6. A pilot scope and change control . . . 15

3. Implementation Guide — Sensitivity Label Taxonomy Design 15

Step 1 — Assess the current label posture (Assess) . . . 16

Step 2 — Design the taxonomy and protection mapping (Design) . . . 16
Step 3 — Build the parent tiers and top-level labels (Configure, -WhatIf first) . . . 16
Step 4 — Build the protected sublabels per the mapping (Configure, -WhatIf first) . . . 17
Step 5 — Publish the taxonomy to the pilot ring (Pilot) . . . 17
Step 6 — Validate structure, marking, and protection (Validate) . . . 17
Step 7 — Expand to broad publish (Expand) . . . 18
5. Testing & Validation Guide — Sensitivity Label Taxonomy Design 18
Test 1 — Taxonomy built exactly as designed . . . 19
Test 2 — Parents group only (never protect, never selectable) . . . 19
Test 3 — Content marking appears (marking-only tiers) . . . 19
Test 4 — Encryption/protection actually enforces (protecting sublabels) . . . 19
Test 5 — Naming convention conformance . . . 20
Test 6 — Pilot scoping is correct . . . 20
Test 7 — Tooltips are present and plain-language . . . 20
Test 8 — Full health check before broad publish . . . 20
6. Troubleshooting — Sensitivity Label Taxonomy Design (Microsoft Purview) 21
Connectivity & licensing . . . 21
Assess & design (Steps 1–2) . . . 21
Building parents & sublabels (Steps 3–4) . . . 21
Publishing to the pilot ring (Step 5) . . . 21
Validation & protection (Step 6) . . . 22
Health: orphans & sprawl (IPTAX-ScopeHealth.ps1) . . . 22
Broad publish (Step 7) . . . 22
What this solution does NOT decide . . . 22
7. Best Practices — Sensitivity Label Taxonomy Design 22
Taxonomy structure . . . 23
Naming convention . . . 23
Protection / marking mapping . . . 23
Piloting & rollout . . . 24
Keeping the taxonomy healthy over time . . . 24
Security & safety hygiene . . . 24
8. Deliverables Summary — Sensitivity Label Taxonomy Design 25
What you receive . . . 25
Documentation . . . 25
PowerShell tools (5) . . . 25
Outcomes . . . 26

Explicitly out of scope (deferred) 27
Appendix A — PowerShell Scripts 28

Executive Summary — Sensitivity Label Taxonomy Design (Microsoft Purview)

The problem

Sensitivity labels are how Microsoft Purview classifies and protects content — but a **bad taxonomy breaks adoption and trust**. Too many labels and users guess or ignore them; too few and the labels don't reflect real sensitivity; inconsistent names produce sprawl no one can maintain. Worse, the taxonomy is **expensive to change after publish** — existing content keeps the label it was stamped with, so a rushed design has to be lived with or painfully rebuilt. Left unaddressed, this is a compliance gap and a data-risk exposure: the investment in Purview fails to deliver classified, protected content.

The solution

A complete, self-guided way to **design a sensitivity label taxonomy users will actually use** — and to build it so it doesn't have to be rebuilt. It delivers a workable **tier structure** (the right number of tiers, with sublabels for nuance), a **stable naming convention** (immutable internal names, plain-language display names and tooltips), and a **protection/marking mapping** — which labels encrypt (rights by group), which mark, and with what header/footer/watermark. The designed taxonomy is then configured in the tenant, **published to a pilot ring**, and validated before broad rollout. It uses only the Microsoft Purview portal and Security & Compliance PowerShell — no third-party tooling.

How it works

The engagement follows one disciplined model: **Assess** → **Design** → **Configure** → **Pilot (test)** → **Validate** → **Expand.** Assess the current label posture and catch name collisions and sprawl; **design** the tiers, sublabels, naming convention, and protection mapping with the governance owners and get them signed off; build the parent tiers and top-level labels, then the protected sublabels per the mapping — each previewed with `-WhatIf` before commit; publish to a **pilot ring**; validate structure, marking, and that encryption actually enforces; then expand to broad publish. Five PowerShell tools — read-only or `-WhatIf` by default — automate discovery, configuration, validation, reporting, and health checks.

What is delivered

- Six implementation documents (Overview, Requirements, Implementation, Testing, Best Practices, Troubleshooting) plus this summary — including the naming convention and the protection/marking mapping template.
- Five Security & Compliance PowerShell tools: posture/collision discovery, taxonomy + publish config helper, design-conformance validation, taxonomy/mapping reporting, and a scope/health helper.
- A named, gated, reversible rollout with explicit pass/fail criteria at every step.

Business outcome

A label taxonomy that **users adopt because it's clear** — the right number of tiers, sublabels that say who content is for, a naming convention that prevents sprawl, and protection applied exactly where it's needed and proven to enforce before it reaches everyone. Classification that sticks, at a fraction of the cost of a bespoke consulting engagement — and a foundation the rest of the Purview program (DLP, auto-labeling, retention, container labeling) builds on.

Important boundaries

The **tier count, the final label names, which tiers encrypt, and the rights model** are **determinations owned by your security, compliance, records, and legal functions** — this solution provides the design method, the mapping template, and the technical configuration and evidence, not the determination. Sensitivity labeling and label-based encryption are broadly licensed, but some advanced behaviors are plan-gated (E5 / E5 Compliance) and the Purview portal is evolving — availability and paths are flagged "verify your licensing/tenant," never assumed.

1. Solution Overview — Sensitivity Label Taxonomy Design (Microsoft Purview)

What this solution is

A **sensitivity label taxonomy** is the ordered set of labels users choose from to classify content in Microsoft Purview — the tier names, the parent/sublabel structure, the naming convention, and the **protection and marking each label applies**. This solution is a repeatable way to **design** that taxonomy well: decide how many tiers you need, structure parents and sublabels, agree a stable naming convention, and produce a **protection/marking mapping** (which labels encrypt, which mark, and with what rights) — then configure the designed taxonomy in the tenant, **publish it to a pilot ring**, and validate that it behaves as designed. All configuration uses only the Microsoft Purview portal and Security & Compliance PowerShell (no third-party tooling), and every high-impact change is piloted in a small ring before broad rollout.

This is the **design** build — the taxonomy, sublabels, naming, and protection/marking *mapping* done deliberately so users actually adopt it. It is distinct from:

- the **full labels implementation** (which also builds service-side auto-labeling and retention labels with adaptive scopes),
- **mandatory vs recommended labeling design** (the default-label / prompt-for-justification decision),
- and **container labeling** for Teams/Groups/Sites — each a separate solution. A well-designed taxonomy is the foundation those build on.

Why this matters

- **A bad taxonomy breaks adoption.** Too many labels and users guess or ignore them; too few and the labels don't reflect real sensitivity. The taxonomy *is* the user-facing guidance — getting the structure and names right is what makes classification stick.

• **Renaming and reordering after publish is disruptive.** Existing content keeps the label it was

stamped with, and a display-name change confuses users. The taxonomy is expensive to change later — so it is worth designing once, deliberately, up front.

• **Protection travels with the label.** When a sensitivity label encrypts, that protection stays

enforced when the file leaves the tenant. Deciding *which* tiers encrypt (and with what rights) is a design decision with real blast radius — encrypt the wrong tier and you break co-authoring and downstream automation.

• **A stable naming convention prevents sprawl.** Consistent, immutable internal names and predictable

display names keep the taxonomy legible as it grows and make every downstream control (DLP, auto-labeling, retention) easier to target.

The label object model (what you are designing)

Property	What it is	Design guidance
DisplayName	What users see in Office (a sublabel shows as `Parent \ Child`)	Plain business language; short; the tooltip carries the detail
Name (internal)	Immutable identifier used by cmdlets/policies	Stable slug; **never reused or renamed**; the naming convention lives here
Tooltip	The only in-context help a user gets	One plain-language sentence: what it means + a handling instruction
Priority / order	Position in the ordered list (least → most sensitive)	The order *is* the guidance; set it deliberately
ParentId	Links a sublabel to its parent tier	Parents group only; **parents must not apply protection** and are not selectable
Protection	Encryption + rights (usage rights per group)	The high-impact decision — map it explicitly (below)
Content marking	Header / footer / watermark text	Visible classification; cheap, low-friction, use broadly

The naming convention (the core design deliverable)

Every label this solution creates follows one convention, reused verbatim across the docs, scripts, and portal. This is what keeps the taxonomy legible and prevents sprawl.

• **Display name (what users see):**

• **Top tier:** the plain sensitivity word — `Public`, `Internal`, `Confidential`, `Highly Confidential`.

• **Sublabel:** the **audience / handling** it targets — shown as `Confidential \ All Employees`,

`Highly Confidential \ Specified People`. The sublabel display name is the *audience*, not a repeat of the tier.

- **Internal name (immutable identifier):** a stable slug `SL-<Tier>[-<Audience>]` —

e.g. `SL-Public`, `SL-Confidential`, `SL-Conf-AllEmp`, `SL-HighlyConf-Specified`. The internal name is **never renamed or reused**; display names can be adjusted, internal names cannot.

- **Order (priority):** ascending by sensitivity — `Public (0) → Internal (1) → Confidential (2) →

Highly Confidential (3)`, sublabels ordered within their parent.

Sample taxonomy (a technical starting point — confirm with your governance owners)

> *This is a **technical starting point**, not a determination. The final tier count, names, protection settings, and rights model are a **compliance/records/security decision** (see §2 deferral). The names below are reused verbatim across the docs and scripts.*

Order	Display name	Internal Name	Parent	Selectable?
0	`Public`	`SL-Public`	—	yes
1	`Internal`	`SL-Internal`	—	yes
2	`Confidential`	`SL-Confidential`	— (grouping only)	**no**
2.1	`Confidential \ All Employees`	`SL-Conf-AllEmp`	`SL-Confidential`	yes
2.2	`Confidential \ Trusted People`	`SL-Conf-Trusted`	`SL-Confidential`	yes
3	`Highly Confidential`	`SL-HighlyConf`	— (grouping only)	**no**
3.1	`Highly Confidential \ All Employees`	`SL-HighlyConf-AllEmp`	`SL-HighlyConf`	yes
3.2	`Highly Confidential \ Specified People`	`SL-HighlyConf-Specified`	`SL-HighlyConf`	yes

The protection / marking mapping (the second core deliverable)

For **each** selectable label, the design records exactly what it applies. This mapping is the artifact that turns a list of names into an enforceable design.

Label	Encryption	Rights (by group)	Content marking
`Public`	none	—	Footer: `Public`
`Internal`	none	—	Footer: `Internal — do not distribute externally`
`Confidential \ All Employees`	Encrypt	all internal users → **Co-Author**	Footer: `Confidential`
`Confidential \ Trusted People`	Encrypt	named group → **Reviewer** (Do-Not-Forward-style)	Footer: `Confidential`
`Highly Confidential \ All Employees`	Encrypt	all internal users → **Co-Author**	Footer: `Highly Confidential` + watermark `HIGHLY CONFIDENTIAL`
`Highly Confidential \ Specified People`	Encrypt	named recipients → **Viewer** (no export/print)	Footer: `Highly Confidential` + watermark

Design rules baked into the mapping:

• **Parents never protect** — `Confidential` and `Highly Confidential` group only; they are not selectable and carry no encryption/marking.

• **Lower tiers mark, don't encrypt** — `Public`/`Internal` get visible footers with **zero friction**.

• **Rights point at groups, not people** — every encrypting label's rights target a **mail-enabled security group**, so membership changes never require editing the label.

• **Reserve encryption for what needs it** — encryption travels with the file and is the strongest control, but it breaks some co-authoring/automation paths; only the tiers that genuinely need it encrypt, and each is piloted first.

Artifact names used in every doc and script

Artifact	Name
Sensitivity label policy (pilot)	`SL Policy - Pilot - Ring1`
Sensitivity label policy (broad)	`SL Policy - Broad - AllUsers`
Pilot ring group (placeholder)	`grp-purview-pilot@contoso.com`
Rights group — all employees (placeholder)	`grp-all-employees@contoso.com`
Rights group — trusted people (placeholder)	`grp-trusted-people@contoso.com`
Rights group — specified people (placeholder)	`grp-specified-people@contoso.com`

What you can control

• The **number of tiers** and whether each has sublabels — kept short enough that users don't guess.

• The **display names, internal names, order, and tooltips** — the whole user-facing structure.

• The **protection/marking mapping** — which labels encrypt, the exact rights per group, and the header/footer/watermark on each.

• **Who the label policy targets** — pilot ring first (`SL Policy - Pilot - Ring1`), then broad.

• Whether the taxonomy is published as a starting design or extended over time — with the naming convention keeping it legible.

Limitations (be honest with buyers)

• **This is design + configuration, not the full label program.** Service-side auto-labeling, retention labels, adaptive scopes, DLP, and container labeling are **separate solutions** — this one delivers a sound taxonomy and its protection mapping, published to a pilot.

• **Licensing gates some protection features.** Manual sensitivity labeling, content marking, and encryption are available broadly (Microsoft 365 E3 tier); some advanced label behaviors are E5-tier — **verify your licensing/tenant**; the solution flags each gate rather than assuming.

• **Protection settings are determinations, not defaults.** The final tier count, rights model, and which tiers encrypt are decisions owned by security/compliance/records (see §2 deferral). This solution provides the design method, the mapping template, and the configuration — not the sign-off.

• **The taxonomy is expensive to change after publish.** Existing content keeps its stamped label; renaming/reordering is disruptive. That is exactly why the design is done deliberately up front.

• **The portal is evolving.** Portal paths and some cmdlet parameters change; where a path is version-sensitive it is flagged "verify in your tenant."

The standard deployment model

> *Assess the current label posture → **design** the taxonomy (tiers, sublabels, naming, protection/ marking mapping) with governance owners → configure the labels (`-WhatIf` → `-Commit`) → **publish to a pilot ring** → validate structure + protection → **expand** to broad publish. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce).***

2. Requirements Guide — Sensitivity Label Taxonomy Design

Verify **every** requirement below before starting the Implementation Guide. These are the **solution-specific** prerequisites for designing and building a sensitivity label taxonomy; they build on the shared **Global Requirements — Microsoft Purview Solutions** (licensing, least-privilege roles, Security & Compliance PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, and a pilot scope with change control). Read the Global Requirements first; this guide does not repeat them.

> ***Deferral note (security / compliance / records).*** *The **tier count, the final label names, which tiers encrypt, and the exact rights model** referenced here are **determinations owned by your security, compliance, records, and legal functions** — not technical defaults. This solution provides the **design method, the mapping template, and the configuration**; it does not decide the classification scheme. Confirm the taxonomy and protection mapping with the accountable owners before publishing.*

2.1 Licensing for sensitivity labels, protection, and content marking

• **Description: Manual sensitivity labeling**, **content marking** (header/footer/watermark), and **encryption** applied through a label are available broadly (the Microsoft 365 E3 tier covers manual sensitivity labeling and label-based encryption). More advanced label behaviors (some auto-labeling, co-authoring on encrypted files at scale, and certain classifier-driven features that other solutions build on) are **E5 / E5 Compliance**-tier — this design solution stays within manual labels + protection + publishing, which E3 covers.

• **Why required:** You cannot create or publish protecting labels without the tenant plan that includes sensitivity labeling and rights protection.

• **How to verify:** M365 admin center → subscriptions/add-ons; confirm **Information Protection** is visible in the Purview portal. **Verify your licensing/tenant** — Microsoft's licensing tables are the authority.

• **How to implement if missing:** Ensure the tenant has the plan covering sensitivity labeling and rights management before building protecting labels.

2.2 Information Protection Admin role (least privilege)

• **Description:** The Purview role group this solution needs is **Information Protection Admin** (create/manage sensitivity labels and label policies). **Global Administrator is not required** for taxonomy work.

• **Why required:** The sensitivity-label cmdlets (`New-Label`, `Set-Label`, `New-LabelPolicy`, `Set-LabelPolicy`) and portal panes are gated by this role group.

• **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm your admin account is in *Information Protection* (or *Information Protection Admin*).

• **How to implement if missing:** Add the admin to the least-privilege role group under change control; wait for role propagation before running cmdlets.

2.3 Security & Compliance PowerShell connectivity

• **Description:** A workstation with the **ExchangeOnlineManagement** module (which provides `Connect-IPPSSession` for Security & Compliance PowerShell) and, where Graph is used for evidence, **Microsoft Graph PowerShell** — with a modern-auth / MFA-capable admin account.

• **Why required:** All five provided scripts and every label/policy cmdlet run through

`Connect-IPPSSession`.

- **How to verify:** `Connect-IPPSSession` succeeds and `Get-Label` returns without error.
- **How to implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm

modern-auth sign-in. See the Global Requirements for the full toolchain.

2.4 A classification scheme and in-scope workloads

- **Description:** An agreed **classification scheme** — how many sensitivity tiers the organization

recognizes and what each means (e.g. `Public` / `Internal` / `Confidential` / `Highly Confidential`) — and the **workloads in scope** (Exchange, SharePoint, OneDrive, Teams; endpoints where client-side labeling applies). The scheme is the input to the taxonomy design; without it the labels are arbitrary.

- **Why required:** The taxonomy structure (tier count, sublabels) and the protection mapping key off the

classification scheme and where labels will be used.

- **How to verify:** Confirm the tier scheme and the in-scope workloads with security/compliance

stakeholders. Check whether any classification scheme already exists (data-classification framework, ISO/regulatory scheme) to align to.

- **How to implement if missing:** Agree the tier scheme (typically 3–5 tiers) before authoring labels;

the Overview's sample taxonomy is a starting point to react to, not a default to adopt blindly.

2.5 A pilot ring and a pilot-first commitment

- **Description:** A named **pilot ring** — a small group of users (e.g. the group backing

`SL Policy - Pilot - Ring1`) — to publish the designed labels to first, and an explicit commitment that **encrypting labels are validated in the pilot** before broad publish (`SL Policy - Broad - AllUsers`).

- **Why required:** Encryption and content-marking behavior is best surfaced cheaply in a small ring;

the house model requires a pilot/test pass before broad enforcement.

- **How to verify:** Confirm the pilot-ring group membership, a change window, and an approver.
- **How to implement if missing:** Create the pilot group and agree the change-control path before

publishing anything.

2.6 Rights-management (encryption) prerequisites

- **Description:** For labels that **encrypt**, the tenant's **Azure Rights Management / unified labeling

protection **service must be active, and you must decide the** rights model** — which mail-enabled security groups get which usage rights (e.g. Co-Author, Reviewer, Viewer) per encrypting label. The Overview's protection mapping is the template for this decision.

- **Why required:** An encrypting label cannot protect if the protection service is not enabled; the

rights definition (points at a group) is a required input to `New-Label -EncryptionEnabled`.

- **How to verify:** Confirm the protection (Rights Management) service is **activated**; confirm each

rights group named in the mapping (`grp-all-employees@…`, `grp-trusted-people@…`, `grp-specified-people@…`) exists and has the intended members.

- **How to implement if missing:** Activate the protection service and create/confirm the mail-enabled

security groups used in the rights definitions before building encrypting labels.

2.7 A signed-off taxonomy and protection-mapping sheet

- **Description:** Before any label is created, a **signed-off design artifact** exists: the tier list

(display names, internal names, order, tooltips), the parent/sublabel structure, and the **protection/marking mapping** (which labels encrypt, the rights per group, the header/footer/ watermark). This is the design deliverable the configuration implements verbatim.

- **Why required:** Renaming or reordering after publish is disruptive (existing content keeps the old

label); the design must be agreed before it is built so it does not have to be rebuilt.

- **How to verify:** The design sheet exists and is signed off by the accountable security/compliance

owner; every internal `Name` is unique and does not collide with an existing label (confirm via `IPTAX-Discovery.ps1`).

- **How to implement if missing:** Run the Design step (Implementation Step 2) with the governance owners

and capture the sheet before creating labels.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Sensitivity Label Taxonomy Design

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`) and **Security & Compliance PowerShell** (`Connect-IPPSSession`). Every write is previewed with `-WhatIf` first, every encrypting label is **piloted in a small ring** before broad publish, and label names follow the naming convention from the Overview and are reused verbatim. Portal paths are current at authoring — **verify in your tenant**, as the portal is evolving.

> ***Model:** Assess the current posture → **design** the taxonomy (tiers, sublabels, naming, protection/ marking mapping) → build the labels (`-WhatIf` → `-Commit`) → publish to a **pilot ring** → validate*

*structure + protection → **expand** to broad publish. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce). Deferral:** tier count, final names, which tiers encrypt, and the rights model are **security/ compliance/records determinations** — confirm the design sheet (§2.7) before building.*

Step 1 — Assess the current label posture (Assess)

1. Run `IPTAX-Discovery.ps1` (read-only). It inventories existing **sensitivity labels** (`Get-Label`), their **parent/sublabel structure and order** (Priority / ParentId), **protection/marking** per label (encryption, header/footer/watermark), and existing **label policies** (`Get-LabelPolicy`) — plus a **naming-convention conformance** view and any **name collisions** with the taxonomy you plan to build.

1. Review the output: what labels already exist, whether there is label sprawl or inconsistent naming, and whether any internal `Name` you plan to use is already taken. This is your baseline.

1. **Gate:** you have a written baseline of every existing label/policy and have confirmed **no internal `Name` collisions** with the taxonomy in the design sheet.

Step 2 — Design the taxonomy and protection mapping (Design)

1. With your security/compliance/records owners, finalize the design using the Overview's sample as a starting point to react to. Record, for **each label**:

• **display name**, **internal** `Name` (per the `SL-<Tier>[-<Audience>]` convention), **tooltip**, and **order**;

• the **parent/sublabel structure** (parents group only, never protect, never selectable);

• the **protection/marking mapping** — encryption yes/no, the rights group + usage-rights level, and the header/footer/watermark text.

1. Confirm the pilot ring (the group backing `SL Policy - Pilot - Ring1`) and the rights groups named in the mapping exist (§2.6).

1. **Gate:** a **signed-off taxonomy + protection-mapping sheet** exists — names, order, structure, protection, and rights — before any label is created (§2.7). Internal names are unique and final.

Step 3 — Build the parent tiers and top-level labels (Configure, -WhatIf first)

1. Run `IPTAX-TaxonomyConfig.ps1 -WhatIf` (the default). It prints exactly which labels it would create with `New-Label`, **in order** — the non-protecting tiers (`SL-Public`, `SL-Internal`) and the **parent grouping labels** (`SL-Confidential`, `SL-HighlyConf`, created with **no protection** and as non-selectable groupings) — **without making changes**.

1. Review every line against the design sheet: display names, internal names, tooltips, order, and that parents carry no protection.

1. Re-run `IPTAX-TaxonomyConfig.ps1 -Commit` to create these labels. They exist but are **not published**

yet — labels only appear to users once a policy publishes them (Step 5).

1. **Gate:** `Get-Label` shows `SL-Public`, `SL-Internal`, `SL-Confidential`, `SL-HighlyConf` in the

correct order; the two parents carry no encryption/marking.

Step 4 — Build the protected sublabels per the mapping (Configure, -WhatIf first)

1. Run `IPTAX-TaxonomyConfig.ps1 -Sublabels -WhatIf`. It prints the `New-Label` calls for each sublabel

with its **parent link**, **content marking**, and **encryption/rights** exactly as the mapping specifies — `SL-Conf-AllEmp`, `SL-Conf-Trusted`, `SL-HighlyConf-AllEmp`, `SL-HighlyConf-Specified` — **without making changes**.

1. Review every line: confirm each encrypting sublabel's rights definition targets the correct

mail-enabled security group (§2.6) and usage-rights level, and that the footer/watermark text matches the mapping.

1. Re-run `IPTAX-TaxonomyConfig.ps1 -Sublabels -Commit` to create the sublabels. Encryption and marking

are set on the labels but **not yet visible to anyone** — nothing is published.

1. **Gate:** `Get-Label` shows every sublabel under its correct parent with the intended protection; the

full ordered taxonomy matches the design sheet. Run `IPTAX-Validation.ps1` to confirm structure/order/ protection == design.

Step 5 — Publish the taxonomy to the pilot ring (Pilot)

1. Run `IPTAX-TaxonomyConfig.ps1 -PublishPolicy -PolicyName "SL Policy - Pilot - Ring1" -PilotGroup "grp-purview-pilot@contoso.com" -WhatIf`.

It shows the `New-LabelPolicy` it would run — publishing the **ordered, selectable** label set (parents excluded from selection) to the pilot group only.

1. Re-run with `-Commit` to create `SL Policy - Pilot - Ring1`.

2. In the Purview portal (**Information Protection** → **Label policies**) confirm the policy is published to

the pilot group and the labels appear in Office for a pilot user (allow time for client sync).

1. **Gate:** pilot users see the full ordered taxonomy in Word/Excel/PowerPoint/Outlook; parents are not

individually selectable; a **non-pilot** user sees nothing. Rollback: `Remove-LabelPolicy "SL Policy - Pilot - Ring1"` unpublishes without deleting the labels.

Step 6 — Validate structure, marking, and protection (Validate)

1. Run `IPTAX-Validation.ps1 -Full`. It re-checks the built taxonomy against the design: order, parent/

sublabel structure, per-label protection/marking, naming-convention conformance, and that the pilot policy is published to the intended scope.

1. As a pilot user, apply a **marking-only** label (`Internal`) and confirm the footer appears; apply an

encrypting sublabel (`Confidential \ Trusted People` or `Highly Confidential \ Specified People`) and confirm protection enforces (Testing Guide §5, Test 4 — open as an unauthorized user → denied).

1. Complete the **Testing Guide (§5)**.
2. **Gate:** all §5 tests pass — structure as designed, marking visible, encryption enforces, scope

correct.

Step 7 — Expand to broad publish (Expand)

1. Only after every §5 gate passes, publish the taxonomy broadly:

`IPTAX-TaxonomyConfig.ps1 -PublishPolicy -PolicyName "SL Policy - Broad - AllUsers" -PilotGroup "<all-users-or-broad-scope>" -WhatIf`, review, then `-Commit`. This publishes the same designed taxonomy to the broad scope.

1. Communicate to users what the labels mean and what changes (marking/encryption they'll see) **before**

the broad publish reaches them — the ticket flood comes from surprise, not from labels.

1. Re-run `IPTAX-Validation.ps1 -Full` and confirm the broad policy is published to the intended scope.
2. **Gate:** broad publish confirmed to the correct scope; a rollback path

(`Remove-LabelPolicy "SL Policy - Broad - AllUsers"`) is documented; the taxonomy design sheet, the built labels, and the published policy all agree.

Order of operations

`Assess posture → Design taxonomy + protection mapping (signed off) → Build parents/top-level (-WhatIf → commit) → Build protected sublabels (-WhatIf → commit) → Publish to pilot ring → Validate structure + marking + protection → Expand to broad publish`

5. Testing & Validation Guide — Sensitivity Label Taxonomy Design

Run these tests after the Implementation Guide, **in order**. Each has an explicit **pass/fail** signal and a rollback. Do not expand from the pilot ring to broad publish until the relevant tests pass. All checks use the Purview portal and Security & Compliance PowerShell — the provided scripts automate the read-only ones.

> ***Reminder:*** *the tier count, names, and protection mapping under test are the security/compliance/records determinations from §2 — you are validating that the built taxonomy *matches the signed-off design and behaves as specified*, not deciding the scheme.*

Test 1 — Taxonomy built exactly as designed

- **How:** `IPTAX-Validation.ps1 -Full`, or `Get-Label | Sort-Object Priority | Format-Table DisplayName,Name,ParentId,Priority`.
- **Pass:** every label from the signed-off design sheet exists — correct display name, internal `Name`, parent link, and **order** (least → most sensitive); no missing or extra labels.
- **Fail:** missing/extra labels, wrong order, or wrong internal name → fix in `IPTAX-TaxonomyConfig.ps1` and re-run. **Do not rename an existing internal** `Name` — create the correct one and retire the wrong one.

Test 2 — Parents group only (never protect, never selectable)

- **How:** `IPTAX-Validation.ps1 -Full`; confirm the parent labels (`SL-Confidential`, `SL-HighlyConf`) report **no encryption and no marking**, and confirm in Office that a pilot user **cannot select the parent directly** — only its sublabels.
- **Pass:** parents carry no protection and are not individually applicable; only sublabels are selectable under them.
- **Fail:** a parent applies protection or is selectable → correct the label so the parent is a grouping only.

Test 3 — Content marking appears (marking-only tiers)

- **How:** as a **pilot** user, apply `Public` and `Internal` to test documents and confirm the footer text appears as designed.
- **Pass:** the configured footer (e.g. `Internal — do not distribute externally`) appears; no encryption is applied to these tiers.
- **Fail:** no marking, wrong text, or unexpected encryption on a marking-only tier → correct the label's content-marking settings and re-test.

Test 4 — Encryption/protection actually enforces (protecting sublabels)

- **How:** as a pilot user, apply `Confidential \ Trusted People` (or `Highly Confidential \ Specified People`) to a test document; attempt to open it as a user **outside** the rights group.
- **Pass:** the unauthorized user is **denied**; the authorized rights (Co-Author vs Reviewer vs Viewer/no-export) behave as the mapping defines; the watermark/footer appears where configured.

• **Fail:** unauthorized access succeeds, or rights are wrong → the protection service or the label's `EncryptionRightsDefinitions` is misconfigured (§2.6). **Do not publish broadly** until fixed.

Test 5 — Naming convention conformance

• **How:** `IPTAX-Reporting.ps1 -ConformanceOnly` (or review the conformance column in the full report).

• **Pass:** every internal `Name` matches the `SL-<Tier>[-<Audience>]` convention; display names and order match the design; no legacy/ad-hoc labels break the scheme in the published set.

• **Fail:** non-conforming names or stray labels → reconcile with the design sheet; retire non-conforming labels rather than renaming them.

Test 6 — Pilot scoping is correct

• **How:** as a **pilot** user, confirm the full taxonomy appears in Office; as a **non-pilot** user, confirm **no** labels appear. `Get-LabelPolicy "SL Policy - Pilot - Ring1"` shows the pilot group.

• **Pass:** pilot users see the full ordered label set; non-pilot users see nothing (publish is scoped).

• **Fail:** labels missing for pilot users (allow client sync time) or visible to non-pilot users (policy scope wrong) → re-check `SL Policy - Pilot - Ring1` targeting.

Test 7 — Tooltips are present and plain-language

• **How:** in Office, hover each label as a pilot user; or `Get-Label | Format-Table DisplayName,Tooltip`.

• **Pass:** every selectable label has a plain-language tooltip that states what it means and gives a handling instruction — the only in-context help a user gets.

• **Fail:** missing or jargon tooltips → set a clear tooltip per label and re-publish.

Test 8 — Full health check before broad publish

• **How:** `IPTAX-Validation.ps1 -Full`.

• **Pass:** structure/order/protection == design, naming conformant, tooltips present, pilot policy scoped correctly — all green.

• **Fail:** any red → resolve before publishing `SL Policy - Broad - AllUsers`.

Exit criteria (all must be true before broad publish)

`Taxonomy built as designed · parents group-only · marking visible · encryption enforces · naming conformant · pilot scope correct · tooltips present · full health check green · rollback documented per policy`

6. Troubleshooting — Sensitivity Label Taxonomy Design (Microsoft Purview)

Common issues, grouped by step. Each has a **symptom**, the **likely cause**, and a **resolution**. Every write in `IPTAX-TaxonomyConfig.ps1` is `-WhatIf` by default and only applies with `-Commit`; building labels does **not** surface them to users (only a policy publish does), and every encrypting label is piloted in a small ring first. `IPTAX-Discovery.ps1`, `IPTAX-Reporting.ps1`, `IPTAX-ScopeHealth.ps1`, and `IPTAX-Validation.ps1` are read-only. Portal paths are current at authoring — the portal evolves, so **verify paths in your tenant**.

Connectivity & licensing

Symptom: `Get-Label` / `New-Label` unavailable. **Likely cause:** `ExchangeOnlineManagement` not connected, or the Information Protection role is missing. **Resolution:** `Install-Module ExchangeOnlineManagement`, `Connect-IPPSSession`, and confirm the Information Protection Admin role. Sensitivity labels require the appropriate compliance licensing — verify your licensing/tenant.

Assess & design (Steps 1–2)

Symptom: `IPTAX-Discovery.ps1` reports an internal-name **collision** with a planned label. **Likely cause:** an existing label already uses that internal `Name`. **Resolution:** choose a different internal name (the `Name` is immutable once created) or retire the old label first. Internal names must be unique and final before building (Step 2 gate).

Symptom: the design sheet isn't signed off but you want to start building. **Likely cause:** tier count, final names, which tiers encrypt, and the rights model are **security/compliance/records determinations** (§2.7). **Resolution:** get the design sheet signed off first — building labels against an unconfirmed design causes rework (labels can't be freely renamed/deleted once applied).

Building parents & sublabels (Steps 3–4)

Symptom: `IPTAX-TaxonomyConfig.ps1` printed calls but created nothing. **Likely cause:** `-WhatIf` (the default) — no `-Commit`. **Resolution:** review the previewed `New-Label` lines against the design sheet, then re-run with `-Commit` (add `-Sublabels` for Step 4).

Symptom: a parent label appears selectable / seems to protect content. **Likely cause:** a parent grouping label was created with protection or as selectable. **Resolution:** parents must be **grouping-only** — no encryption, not individually selectable. Rebuild the parent per the design (parents group; sublabels protect). Confirm with `IPTAX-Validation.ps1`.

Symptom: an encrypting sublabel's rights don't target the right people. **Likely cause:** the rights definition points at the wrong or a non-mail-enabled security group. **Resolution:** confirm the **mail-enabled security group** named in the mapping (§2.6) exists and is correct; re-preview with `-WhatIf` before `-Commit`.

Publishing to the pilot ring (Step 5)

Symptom: pilot users don't see the labels. **Likely cause:** client sync delay, the policy didn't publish to the pilot group, or the user isn't in the group. **Resolution:** allow time for Office client sync; confirm with

`IPTAX-ScopeHealth.ps1 -Group <pilot-group>` that the group has the intended members and the policy publishes the labels; verify in **Information Protection** → **Label policies**.

Symptom: a **non-pilot** user sees the labels. **Likely cause:** the policy scope is broader than the pilot group. **Resolution:** correct the policy's published scope to the pilot group only; rollback is `Remove-LabelPolicy "SL Policy - Pilot - Ring1"` (unpublishes without deleting labels).

Validation & protection (Step 6)

Symptom: applying an encrypting sublabel doesn't protect content / everyone can still open it. **Likely cause:** the protection (Azure Rights Management) service isn't active, or the label isn't actually encrypting. **Resolution:** run `IPTAX-ScopeHealth.ps1` — it checks `Get-IRMConfiguration`; if Rights Management licensing is off, enable it (verify your licensing/tenant). Confirm the label's `EncryptionEnabled` in `IPTAX-Reporting.ps1`. Test as an unauthorized user → access denied.

Symptom: `IPTAX-Reporting.ps1` conformance flags labels that don't match `SL-`. **Likely cause:** pre-existing labels use a different naming convention. **Resolution:** decide whether to rename (new label, since `Name` is immutable) or accept legacy names; keep the taxonomy design sheet as the authority. Use `-ConformanceOnly` for just the conformance view.

Health: orphans & sprawl (IPTAX-ScopeHealth.ps1)

Symptom: a policy shows "publishes NO labels (orphaned)". **Likely cause:** a label policy exists with no labels attached. **Resolution:** attach the intended labels or remove the empty policy.

Symptom: labels flagged "in no policy (sprawl)". **Likely cause:** labels were built but never published — expected for **parent grouping** labels, a problem for a leaf label you meant to publish. **Resolution:** publish the leaf labels that should be user-visible; parent groupings are intentionally not individually published/selectable.

Broad publish (Step 7)

Symptom: a ticket flood after broad publish. **Likely cause:** users were surprised by new marking/encryption. **Resolution:** communicate what the labels mean and what changes **before** the broad publish reaches users. Rollback: `Remove-LabelPolicy "SL Policy - Broad - AllUsers"` unpublishes without deleting labels.

What this solution does NOT decide

The **number of tiers**, the **final names**, **which tiers encrypt**, and the **rights model** are **security / compliance / records determinations** (design sheet §2.7). This solution designs, builds, pilots, validates, and publishes the taxonomy per that signed-off design — it does not make the classification policy determination.

7. Best Practices — Sensitivity Label Taxonomy Design

Field-tested practices for designing a label taxonomy users will **actually use** — and for building it so it doesn't have to be rebuilt. These complement the Implementation and Testing guides.

Taxonomy structure

• **Keep it short.** Three to five top-level tiers is the practical ceiling — more and users guess or ignore them. Use **sublabels** (parent + children) to add nuance without lengthening the top list.

• **Order by sensitivity, least → most.** Users pick from the ordered list; the order *is* the guidance. Set Priority deliberately and never leave ordering to chance.

• **Parents group only — never protect, never select.** A parent (`Confidential`, `Highly Confidential`) exists to group its sublabels. Give it no encryption/marking and keep it non-selectable; the user always picks a specific sublabel with a defined handling.

• **Design once, with governance owners.** Renaming or reordering after publish is disruptive (existing content keeps the old label, users get confused). Get the taxonomy and protection mapping **signed off before** creating any label (§2.7).

Naming convention

• **Separate display names from internal names.** Display names are for users and can be tuned; internal `Name` values are immutable identifiers cmdlets and policies bind to — **never rename or reuse** an internal name. Follow one convention (`SL-<Tier>[-<Audience>]`) so the set stays legible as it grows.

• **Name sublabels by audience/handling, not by repeating the tier.** `Confidential \ Trusted People` tells the user *who* it's for; `Confidential \ Confidential-2` tells them nothing.

• **Write plain-language tooltips.** The tooltip is the only in-context help a user gets. "Confidential — share only with named colleagues; do not send outside the company" beats a policy code.

• **Retire, don't rename.** If a name is wrong, create the correct label and retire the wrong one — a rename ripples through everything already stamped or bound to that name.

Protection / marking mapping

• **Map protection explicitly, per label.** A taxonomy isn't done until every selectable label has a recorded decision: encrypt (yes/no), rights group + usage-rights level, and header/footer/watermark. The mapping is what turns a list of names into an enforceable design.

• **Encrypt deliberately.** Encryption travels with the file and is the strongest control — but it also breaks co-authoring in some paths, third-party tools, and downstream automation. Encrypt only the tiers that truly need it, and always pilot first.

• **Prefer marking for the lower tiers.** `Public`/`Internal` should mark (footer) but not encrypt — you get visible classification with zero friction and no support load.

• **Define rights by group, not by person.** Point `EncryptionRightsDefinitions` at mail-enabled

security groups so membership changes don't require editing the label.

- **Test the "unauthorized open" every time.** The only proof encryption works is that someone *outside*

the rights is denied. Make that Test 4 non-negotiable before broad publish.

Piloting & rollout

- **Pilot ring, always.** Publish to `SL Policy - Pilot - Ring1` before `SL Policy - Broad - AllUsers`.

The pilot surfaces marking, co-authoring, and encryption issues cheaply, on a small group.

- **Document a rollback per policy.** `Remove-LabelPolicy "<name>"` unpublishes without deleting the

labels — know the exact command before you publish. Removing a **policy** never deletes the underlying **labels**, so rollback is low-risk and reversible.

- **Communicate before broad rollout.** Tell users what the labels mean, what changes (marking/

encryption they'll see), and where to get help — surprise, not labels, generates tickets.

- **Publish selectable labels only.** Exclude parents from the policy's applicable set so users can't

pick a bare grouping label.

Keeping the taxonomy healthy over time

- **Re-baseline periodically.** Run `IPTAX-Discovery.ps1` on a cadence to catch label sprawl, orphaned

policies, ad-hoc labels that break the naming convention, and drift from the design sheet.

- **Report conformance.** `IPTAX-Reporting.ps1` exports the taxonomy + protection mapping and flags any

label that doesn't conform to the naming convention — use it as the living record of the design.

- **Change under control.** Any addition to the taxonomy goes through the same design → sign-off → `-WhatIf` → pilot → validate flow; don't hand-edit labels in the portal outside change control.

- **Keep least-privilege roles. Information Protection Admin** is enough for taxonomy work; don't

grant Global Admin for day-to-day label operations.

Security & safety hygiene

- **No credentials in code.** All scripts authenticate interactively via `Connect-IPPSSession` / `Connect-MgGraph`; never embed secrets. Store any exports securely.

- **Read-only or `-WhatIf` by default.** Every provided script defaults to no-change; writes require an explicit `-Commit`. Keep it that way.

- **Placeholders are placeholders.** The sample tier names, rights groups (`contoso.com`), and tooltips

are starting points — replace every one with your signed-off values before committing.

- **Verify licensing/tenant.** Sensitivity labeling and label-based encryption are broadly licensed, but the portal evolves and some behaviors are plan-gated — flag, don't assume.

8. Deliverables Summary — Sensitivity Label Taxonomy Design

What you receive

A complete, self-guided package to **design and build a sensitivity label taxonomy** in Microsoft Purview — the tier structure, sublabels, a stable naming convention, and the protection/marking mapping — then configure it in the tenant, publish it to a pilot ring, and validate that it behaves as designed. It uses only the Microsoft Purview portal and Security & Compliance PowerShell (no third-party tooling).

Documentation

Document	What it gives you
Solution Overview	The label object model, the naming convention, the sample taxonomy, the protection/marking mapping, what you control, limitations, and the deployment model.
Requirements Guide	Solution-specific prerequisites (licensing for labels/protection, Information Protection Admin role, S&C PowerShell, classification scheme, pilot ring, encryption/rights prerequisites, a signed-off design sheet) on top of the Purview Global Requirements.
Implementation Guide	Ordered steps: assess posture → design taxonomy + mapping → build parents/top-level (`-WhatIf`→commit) → build protected sublabels (`-WhatIf`→commit) → publish to pilot ring → validate → expand to broad publish. Each step named, gated, and reversible.
Testing & Validation Guide	8 pass/fail tests — taxonomy built as designed, parents group-only, marking visible, encryption enforces, naming conformance, pilot scoping, tooltips, and a full health check.
Best Practices	Structure, naming, protection/marking mapping, piloting/rollout, taxonomy-health, and safety guidance.
Troubleshooting Guide	Common failure modes (labels not visible, parent selectable, encryption not enforcing, naming drift, name collisions) with resolutions.

PowerShell tools (5)

All run through `Connect-IPPSSession` (Security & Compliance PowerShell); read-only or `-WhatIf` by default, writes gated behind `-Commit`. No credentials in code.

Script	Role	Default
`IPTAX-Discovery.ps1`	**Posture assessment** — inventory existing sensitivity labels, their parent/sublabel structure and order, protection/marking per label, and label policies; flag naming-convention drift and internal-name collisions before you build.	Read-only
`IPTAX-TaxonomyConfig.ps1`	**Taxonomy + publish config helper** — create the designed parent tiers and top-level labels, create protected sublabels per the mapping, and publish the selectable set to a pilot/broad scope.	`-WhatIf`
`IPTAX-Validation.ps1`	**Design-conformance validator** — check the built taxonomy against the design: order, parent/sublabel structure, per-label protection/marking, and that the pilot policy is scoped correctly.	Read-only
`IPTAX-Reporting.ps1`	**Taxonomy + mapping report** — export the full taxonomy and protection/marking mapping (and a naming-convention conformance view) as the living record of the design.	Read-only
`IPTAX-ScopeHealth.ps1`	**Scope/health helper** — preview the pilot/broad publish scope and its group membership, confirm the protection service is active, and surface orphaned policies and label sprawl.	Read-only

Outcomes

• A **signed-off label taxonomy** built in the tenant — the right number of tiers, clear sublabels, and a stable naming convention that prevents sprawl.

• A **protection/marking mapping** applied per label — the correct tiers encrypt (rights by group), the lower tiers mark, and parents group only.

• The taxonomy **published to a pilot ring**, validated (structure, marking, encryption enforcement), and then broadly — with encryption proven to enforce before broad rollout.

• Repeatable read-only tooling to re-baseline, report, and health-check the taxonomy over time.

Explicitly out of scope (deferred)

• **The classification determination** — the tier count, the final names, which tiers encrypt, and the rights model are owned by security/compliance/records; this solution provides the design method, the mapping template, and the configuration, not the sign-off.

• **Service-side auto-labeling, retention labels, and adaptive scopes** — the deeper labels implementation; **DLP, insider risk, communication compliance**, and **container labeling** for Teams/Groups/Sites; and **mandatory vs recommended (default-label) design** — each a separate Purview solution that a sound taxonomy is the foundation for.

• Any capability your **licensing/tenant** does not include — verify before relying on advanced label behaviors.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPTAX-Discovery.ps1

IPTAX-Reporting.ps1

Purpose: The living record of the designed taxonomy. Reads every sensitivity label and produces the ordered taxonomy with, per label: parent/sublabel, order (Priority), encryption yes/no + rights, content marking (header/footer/watermark), and internal-name convention conformance — so the built labels can be compared to the design sheet at any time. Read-only: reads labels and writes only a local report CSV; it changes no label and publishes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPTAX-Reporting.ps1 -CsvPath .\IPTAX-Taxonomy-Report.csv powershell -ExecutionPolicy Bypass -File .\IPTAX-Reporting.ps1 -ConformanceOnly Requires ExchangeOnlineManagement (Connect-IPPSSession). Role: Information Protection reader. PowerShell 5.1+ or 7+. EXPECTED RESULTS The ordered taxonomy + protection/marking mapping table (and optional CSV), plus a naming-conformance view flagging any internal name that doesn't match the SL- convention. -ConformanceOnly prints just the conformance view. .NOTES Purview — Sensitivity Label Taxonomy Design. Read-only. Which tiers encrypt and the rights model are security/compliance/records determinations (design sheet §2.7) — this report records them, it does not decide them. > [CmdletBinding()] param([string]$CsvPath, [string]$NamePrefix = 'SL-', [switch]$ConformanceOnly, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\IPTAX-Reporting.ps1 -CsvPath .\IPTAX-Taxonomy-Report.csv powershell -ExecutionPolicy Bypass -File .\IPTAX-Reporting.ps1 -ConformanceOnly Requires ExchangeOnlineManagement (Connect-IPPSSession). Role: Information Protection reader. PowerShell 5.1+ or 7+. EXPECTED RESULTS The ordered taxonomy + protection/marking mapping table (and optional CSV), plus a naming-conformance view flagging any internal name that doesn't match the SL- convention. -ConformanceOnly prints just the conformance view. .NOTES Purview — Sensitivity Label Taxonomy Design. Read-only. Which tiers encrypt and the rights model are security/compliance/records determinations (design sheet §2.7) — this report records them, it does not decide them. > [CmdletBinding()] param([string]$CsvPath, [string]$NamePrefix = 'SL-', [switch]$ConformanceOnly, [string]$UserPrincipalName)

IPTAX-ScopeHealth.ps1

Purpose: Pre-publish/health helper. Reports each sensitivity-label policy and the labels it publishes; checks a named pilot/broad group's membership so you publish to the intended scope; confirms the protection (Azure Rights Management) service is active for encrypting labels; and surfaces health issues — policies that publish no labels (orphaned) and labels in no policy (sprawl / never surfaced to users). Read-only: makes no changes and publishes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPTAX-ScopeHealth.ps1 powershell -ExecutionPolicy Bypass -File .\IPTAX-ScopeHealth.ps1 -Group grp-purview-pilot@contoso.com Requires ExchangeOnlineManagement (Connect-IPPSSession); -Group membership + protection-service check also use Connect-ExchangeOnline. PowerShell 5.1+ or 7+. EXPECTED RESULTS Per policy: enabled state + labels published; the named group's member count; protection-service state (or "verify your licensing/tenant"); and lists of orphaned policies and unpublished (sprawl) labels. Resolve orphans/sprawl and confirm scope before broad publish. .NOTES

Purview — Sensitivity Label Taxonomy Design. Read-only. The protection-service and group checks are licensing-/tenant-dependent — items that cannot be confirmed are flagged "verify your licensing/tenant". > [CmdletBinding()] param([string]$Group, # a pilot/broad group to check membership for (e.g. grp-purview-pilot@contoso.com) [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\IPTAX-ScopeHealth.ps1 powershell -ExecutionPolicy Bypass -File .\IPTAX-ScopeHealth.ps1 -Group grp-purview-pilot@contoso.com Requires ExchangeOnlineManagement (Connect-IPPSSession); -Group membership + protection-service check also use Connect-ExchangeOnline. PowerShell 5.1+ or 7+. EXPECTED RESULTS Per policy: enabled state + labels published; the named group's member count; protection-service state (or "verify your licensing/tenant"); and lists of orphaned policies and unpublished (sprawl) labels. Resolve orphans/sprawl and confirm scope before broad publish. .NOTES Purview — Sensitivity Label Taxonomy Design. Read-only. The protection-service and group checks are licensing-/tenant-dependent — items that cannot be confirmed are flagged "verify your licensing/tenant". > [CmdletBinding()] param([string]$Group, # a pilot/broad group to check membership for (e.g. grp-purview-pilot@contoso.com) [string]$UserPrincipalName)

IPTAX-TaxonomyConfig.ps1

IPTAX-Validation.ps1